

Addepar Integration Report for Fynla

Date: 6 March 2026 **Version:** 1.0 **Status:** Feasibility & Architecture Specification

1. Executive Summary

This report details how Fynla can integrate with the Addepar wealth management platform via its REST API. The integration would allow Fynla to import portfolio holdings, account valuations, performance data, and transaction history from Addepar, enriching Fynla's existing financial planning calculations with live custodial data.

The integration is **read-heavy** — Fynla would primarily pull data from Addepar to populate its Investment, Savings, and Retirement modules. Limited write-back (entity creation, custom attribute updates) is feasible but secondary.

Key benefits:

- Automated portfolio data import (eliminating manual entry of holdings, valuations, and transactions)
- Real-time net worth calculations using custodial source data
- Performance metrics (TWR) from Addepar feeding into Fynla's retirement projections
- Unified household view combining Addepar's ownership hierarchy with Fynla's joint ownership model
- Fee transparency using Addepar transaction-level fee data

2. Authentication & Connection Architecture

2.1 Credential Storage

Addepar uses HTTP Basic Auth with an API Key + Secret pair. Fynla must store these securely.

New database table: `addepar_connections`

Column	Type	Purpose
<code>id</code>	bigint	Primary key
<code>user_id</code>	foreignId	Fynla user who owns this connection
<code>firm_id</code>	string	Addepar firm identifier (required header)
<code>api_key</code>	text (encrypted)	Addepar API key
<code>api_secret</code>	text (encrypted)	Addepar API secret
<code>base_url</code>	string	Firm-specific Addepar URL (e.g. <code>https://firm.addepar.com</code>)
<code>is_sandbox</code>	boolean	Whether using development environment

Column	Type	Purpose
last_sync_at	datetime	Last successful data pull
sync_status	enum	idle, syncing, error
sync_error_message	text, nullable	Last error detail
created_at / updated_at	timestamps	Standard

Encryption: Use Laravel's `Crypt::encryptString()` — consistent with Fynla's existing pattern for `account_number`, `sort_code`, and `mortgage_account_number` fields.

2.2 Backend Service Structure

```
app/Services/Addepar/  
  AddeparClient.php           -- HTTP client wrapper (auth headers,  
rate limiting, error handling)  
  AddeparSyncService.php      -- Orchestrates full sync workflow  
  AddeparEntityMapper.php     -- Maps Addepar entities to Fynla models  
  AddeparPortfolioMapper.php  -- Maps portfolio/holdings data  
  AddeparTransactionMapper.php -- Maps transaction history  
  AddeparAttributeResolver.php -- Resolves custom attributes (risk, age,  
etc.)
```

New controller: `app/Http/Controllers/Api/AddeparController.php`

Endpoint	Method	Purpose
/api/addepar/connect	POST	Store credentials, test connection
/api/addepar/disconnect	DELETE	Remove connection
/api/addepar/sync	POST	Trigger manual sync
/api/addepar/status	GET	Get sync status and last sync time
/api/addepar/mapping	GET	Get current entity-to-account mapping
/api/addepar/mapping	PUT	Update entity-to-account mapping

2.3 Request Pattern

Every Addepar API request requires three headers:

```
// AddeparClient.php  
private function buildHeaders(): array  
{  
    return [  
        'Authorization' => 'Basic ' . base64_encode($this->apiKey . ':' .  
$this->apiSecret),  
        'Addepar-Firm' => $this->firmId,  
    ]  
}
```

```
      'Content-Type' => 'application/vnd.api+json',  
    ];  
  }
```

Rate limiting: Addepar does not publish fixed limits. Implement exponential backoff with a conservative default of 60 requests/minute. Log 429 responses and adjust dynamically.

Timeout: Addepar enforces a 60-second timeout. For large portfolio queries, use pagination.

3. Data Model Mapping

3.1 Entity Hierarchy Mapping

Addepar's ownership hierarchy maps to Fynla's models as follows:

Addepar Entity	model_type	Fynla Model	Notes
Household	HOUSEHOLD_NODE	Household	Maps to Fynla's couple/family grouping
Client (Person)	PERSON_NODE	User	Individual client profile
Legal Entity	LEGAL_ENTITY_NODE	Trust	Trusts, companies, partnerships
Account	ACCOUNT_NODE	InvestmentAccount / SavingsAccount / DCPension	Depends on account classification
Investment	INVESTMENT_NODE	Holding	Individual security positions

3.2 Account Type Classification

Addepar accounts need classification into Fynla's account type enums. This requires a **mapping table** because Addepar uses custom attributes for account classification rather than fixed types.

New database table: `addepar_account_mappings`

Column	Type	Purpose
<code>id</code>	bigint	Primary key
<code>user_id</code>	foreignId	Owner
<code>addepar_entity_id</code>	string	Addepar entity ID
<code>addepar_entity_name</code>	string	Display name from Addepar
<code>addepar_entity_type</code>	string	Addepar model_type

Column	Type	Purpose
<code>fynla_model_type</code>	string	Target: <code>InvestmentAccount</code> , <code>SavingsAccount</code> , <code>DCPension</code>
<code>fynla_model_id</code>	bigint, nullable	Linked Fynla record (null = not yet mapped)
<code>fynla_account_type</code>	string, nullable	Mapped account type enum value
<code>auto_sync</code>	boolean	Whether to auto-update on sync
<code>last_synced_at</code>	datetime	Last successful sync for this entity

Recommended auto-classification rules (configurable):

Addepar Account Attribute	Fynla Account Type	Fynla Model
Contains "ISA" or "Individual Savings"	<code>stocks_and_shares_isa</code>	<code>InvestmentAccount</code>
Contains "SIPP" or "Self-Invested"	<code>sipp</code>	<code>InvestmentAccount</code>
Contains "Workplace Pension"	<code>workplace_pension</code>	<code>DCPension</code>
Contains "Cash" / low-risk only	<code>cash_account</code>	<code>SavingsAccount</code>
Contains "GIA" or "General Investment"	<code>stocks</code>	<code>InvestmentAccount</code>
Contains "VCT"	<code>vct</code>	<code>InvestmentAccount</code>
Contains "EIS"	<code>eis</code>	<code>InvestmentAccount</code>
Contains "Bond" (investment)	<code>bond</code>	<code>InvestmentAccount</code>
Unclassified	<code>other</code>	<code>InvestmentAccount</code>

Users must review and confirm mappings on first sync via a dedicated mapping UI.

3.3 Holdings Mapping

Addepar's Positions API maps to Fynla's polymorphic `Holding` model:

Addepar Field	Fynla <code>Holding</code> Field	Notes
<code>direct_owner</code> entity name	<code>holdable_id</code> / <code>holdable_type</code>	Resolved via account mapping
Investment entity <code>original_name</code>	<code>security_name</code>	
Custom attribute (ticker)	<code>ticker</code>	Requires custom attribute lookup
Custom attribute (ISIN)	<code>isin</code>	Requires custom attribute lookup
<code>value</code>	<code>current_value</code>	Position market value
<code>units</code>	<code>quantity</code>	Number of units/shares

Addepar Field	Fynla Holding Field	Notes
<code>unit_price</code>	<code>current_price</code>	Per-unit market price
Calculated from position attributes	<code>allocation_percent</code>	Value / total account value * 100
Custom attribute (asset class)	<code>asset_type</code>	Map Addepar classifications to Fynla enums
Custom attribute (OCF/TER)	<code>ocf_percent</code>	Ongoing Charges Figure
Custom attribute (yield)	<code>dividend_yield</code>	
Position cost basis attribute	<code>cost_basis</code>	If available
Transaction-derived	<code>purchase_price</code> , <code>purchase_date</code>	From transaction history

3.4 User Profile Mapping

Addepar Field	Fynla User Field	Sync Direction
<code>original_name</code> (PERSON_NODE)	<code>first_name</code> , <code>surname</code>	Addepar -> Fynla (initial only)
Custom: <code>_custom_date_of_birth</code>	<code>date_of_birth</code>	Addepar -> Fynla
Custom: <code>_custom_risk_profile</code>	<code>RiskProfile.risk_level</code>	Addepar -> Fynla
Custom: <code>_custom_marital_status</code>	<code>marital_status</code>	Addepar -> Fynla
<code>currency_factor</code>	N/A	Fynla is GBP-only; validate = "GBP"

Important: Addepar stores risk profile and age as **custom attributes** (prefixed `_custom_`). The `AddeparAttributeResolver` service must discover which custom attribute keys the firm uses for these fields. This requires a one-time configuration step per firm.

3.5 Ownership Mapping

Addepar's ownership model is hierarchical (positions between entities with ownership percentages). Fynla uses a single-record joint ownership pattern.

Mapping logic:

```

Addepar: Account --[position, 60%]--> Client A
         Account --[position, 40%]--> Client B

Fynla:   InvestmentAccount {
          user_id: Client A (mapped),
          joint_owner_id: Client B (mapped),
          ownership_type: 'joint',
          ownership_percentage: 60.00 // Client A's share
        }

```

The **AddeparEntityMapper** must:

- 1. Identify all positions linking accounts to PERSON_NODE entities
- 2. Determine primary vs secondary owner (higher percentage = primary)
- 3. Set **ownership_percentage** to primary owner's share
- 4. Map secondary owner to **joint_owner_id**
- 5. Handle trust ownership by setting **trust_id** and **ownership_type: 'trust'**

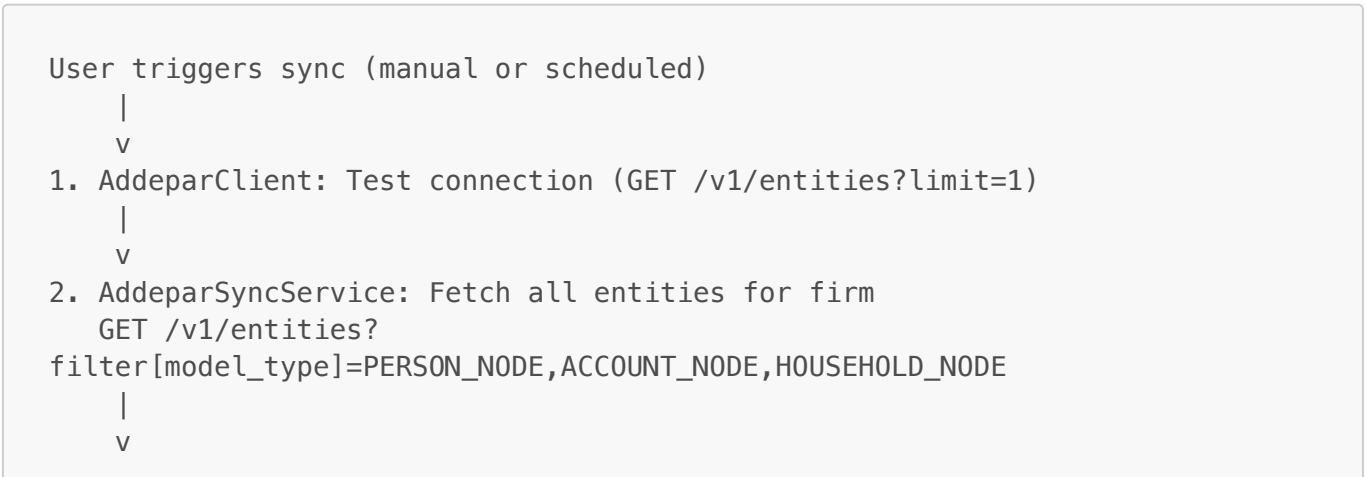
3.6 Transaction Mapping

Addepar's Transactions API provides historical activity that can enrich Fynla's data:

Addepar Transaction Field	Fynla Usage	Target
Buy/Sell transactions	Holding.purchase_price, purchase_date	Cost basis calculation
Contribution transactions	InvestmentAccount.contributions_ytd	YTD contribution tracking
Fee transactions	InvestmentAccount.platform_fee_amount	Actual fee calculation
Dividend transactions	Holding.dividend_yield	Yield calculation
ISA subscription amounts	InvestmentAccount.isa_subscription_current_year	ISA allowance tracking
Pension contributions	DCPension.monthly_contribution_amount	Annual Allowance tracking

4. Sync Workflow

4.1 Full Sync Process



3. AddeparEntityMapper: Match entities to existing Fynla records
 - By addepar_account_mappings table (existing links)
 - By name/account number matching (new entities)
 - Flag unmatched for manual review

|

v
4. AddeparSyncService: Fetch positions for each mapped account
GET /v1/entities/{account_id}/positions

|

v
5. AddeparPortfolioMapper: Update Fynla holdings
 - Create/update Holding records
 - Update InvestmentAccount.current_value (sum of positions)
 - Update DCPension.current_fund_value (if pension account)
 - Update SavingsAccount.current_balance (if cash account)

|

v
6. AddeparSyncService: Fetch portfolio performance (optional)
POST /v1/portfolio/query (TWR, asset allocation)

|

v
7. AddeparTransactionMapper: Sync recent transactions (optional)
GET /v1/transactions?filter[start_date]=...

|

v
8. Post-sync: Trigger Fynla recalculations
 - NetWorthService recalculation
 - Risk profile observer triggers
 - Goal progress updates
 - IHT recalculation (estate values changed)

|

v
9. Update addepar_connections.last_sync_at

4.2 Incremental Sync

For efficiency, use `modified_at` filtering:

```
GET /v1/entities?filter[modified_since]={last_sync_at}
```

Only re-fetch positions for entities modified since last sync. Reduces API calls from $O(n)$ to $O(\text{changed})$.

4.3 Sync Scheduling

Frequency	Trigger	Scope
On-demand	User clicks "Sync" button	Full sync
Daily (recommended)	Laravel scheduler	Incremental sync
On login	After authentication	Quick balance check

Frequency	Trigger	Scope
Post-trade	Webhook (if Addepar supports)	Position update

Implementation: Laravel scheduled command `php artisan addepar:sync` running via cron.

5. Module-Specific Integration Details

5.1 Investment Module

Primary integration point. Addepar is fundamentally a portfolio management platform.

Fynla Feature	Addepar Data Source	API Endpoint
Account list & values	Entities (ACCOUNT_NODE)	GET /v1/entities
Individual holdings	Positions	GET /v1/entities/{id}/positions
Asset allocation	Portfolio query (grouped by asset_class)	POST /v1/portfolio/query
Performance (TWR)	Portfolio performance attributes	POST /v1/portfolio/query
Fee analysis	Transaction fees + custom attributes	GET /v1/transactions
ISA tracking	Custom attribute or transaction sums	Derived
Rebalancing data	Current vs target allocation	Positions + custom attributes

Fynla services impacted:

- **InvestmentAgent** — enhanced with Addepar data source
- **PortfolioAnalyzer** — can use Addepar's TWR instead of calculating locally
- **FeeAnalyzer** — actual fees from transactions vs estimated from rates
- **AssetAllocationService** — real allocation data vs user-entered

5.2 Retirement Module

Pension accounts in Addepar can feed DC pension data.

Fynla Feature	Addepar Data Source
DC pension fund value	Account entity value (pension-type accounts)
Pension holdings	Positions within pension accounts
Contribution tracking	Contribution transactions
Growth rate (actual)	TWR performance data
Fee impact	Transaction-level fee data

Note: DB pensions and State Pension are not held in Addepar — these remain manual entry in Fynla.

5.3 Savings Module

Cash accounts and money market funds in Addepar can map to savings.

Fynla Feature	Addepar Data Source
Cash ISA balances	Cash-type account entities
Interest earned	Transaction history (interest credits)
Emergency fund tracking	Tagged accounts via custom attributes

Limitation: Addepar is investment-focused. Most savings accounts (high street bank accounts, premium bonds) are unlikely to be in Addepar. This integration is secondary.

5.4 Estate Module

Net worth aggregation benefits from accurate Addepar valuations.

Fynla Feature	Addepar Data Source
Investment values for IHT	Account current values
Trust-held investments	Legal entity positions
Gift of investment assets	Transaction history (transfers)
Business Relief qualifying	Custom attribute tagging

IHT calculation chain:

- 1. Addepar sync updates `InvestmentAccount.current_value`
- 2. `NetWorthService` recalculates total assets
- 3. `IHTCalculator` uses updated values for estate tax projection
- 4. Trust-held assets flow through `Trust.total_asset_value`

5.5 Net Worth & Coordination

The most impactful integration for end users.

Addepar sync automatically updates the data sources that feed `NetWorthService`:

- `InvestmentAccount.current_value` (from Addepar positions)
- `DCPension.current_fund_value` (from Addepar pension accounts)
- `SavingsAccount.current_balance` (from Addepar cash accounts, if applicable)

The `CrossModuleAssetAggregator` requires no changes — it already reads from these models. Updated models = updated net worth.

5.6 Goals Module

Goals linked to investment or savings accounts benefit from accurate valuations:

- `Goal.current_amount` can be auto-updated from linked account balances
- `Goal.linked_investment_account_id` -> `InvestmentAccount` -> Addepar sync

- `Goal.linked_savings_account_id` -> SavingsAccount -> Addepar sync
- Milestone tracking becomes more accurate with real-time data

5.7 Risk Module

Addepar custom attributes can feed Fynla's risk assessment:

Addepar Custom Attribute	Fynla Target
<code>_custom_risk_profile</code>	<code>RiskProfile.risk_level</code>
<code>_custom_risk_tolerance</code>	<code>RiskProfile.risk_tolerance</code>
<code>_custom_capacity_for_loss</code>	<code>RiskProfile.capacity_for_loss_percent</code>
<code>_custom_time_horizon</code>	<code>RiskProfile.time_horizon_years</code>

Bidirectional potential: Fynla's risk assessment (calculated from questionnaire + financial position) could be written back to Addepar as a custom attribute, keeping both systems aligned.

6. Frontend Implementation

6.1 New Vue Components

<code>resources/js/components/Addepar/</code>	
<code>AddeparConnectionCard.vue</code>	-- Settings page: connect/disconnect
<code>AddeparSyncStatus.vue</code>	-- Sync status indicator with last sync
<code>time</code>	
<code>AddeparAccountMapping.vue</code>	-- Map Addepar entities to Fynla account
<code>types</code>	
<code>AddeparMappingRow.vue</code>	-- Individual mapping row (entity ->
<code>account type dropdown)</code>	
<code>AddeparSyncModal.vue</code>	-- Sync progress modal with step
<code>indicators</code>	
<code>AddeparImportPreview.vue</code>	-- Preview imported data before
<code>confirming</code>	

6.2 Vuex Store

<code>// store/modules/addepar.js</code>	
<code>const state = {</code>	
<code> connection: null,</code>	<code>// { firm_id, is_sandbox, last_sync_at,</code>
<code> sync_status }</code>	<code>sync_status }</code>
<code> mappings: [],</code>	<code>// addepar_account_mappings</code>
<code> syncProgress: null,</code>	<code>// { step, total_steps, current_entity,</code>
<code> progress_percent }</code>	<code>progress_percent }</code>
<code> loading: false,</code>	
<code> error: null,</code>	
<code>};</code>	

6.3 API Service

```
// services/addeparService.js
const addeparService = {
  async connect(credentials) { return (await api.post('/addepar/connect',
credentials)).data; },
  async disconnect() { return (await
api.delete('/addepar/disconnect')).data; },
  async sync() { return (await api.post('/addepar/sync')).data; },
  async getStatus() { return (await api.get('/addepar/status')).data; },
  async getMappings() { return (await api.get('/addepar/mapping')).data;
},
  async updateMappings(mappings) { return (await
api.put('/addepar/mapping', { mappings })).data; },
};
```

6.4 UI Location

- **Settings page** (`SettingsView.vue`): New "Data Connections" section with Addepar connection card
- **Investment dashboard**: "Last synced" indicator on account cards
- **Net worth dashboard**: "Addepar-linked" badge on synced accounts
- **Account detail views**: Source indicator (manual vs Addepar-synced)

6.5 Preview Mode Consideration

All Addepar write endpoints must be added to `PreviewWriteInterceptor::EXCLUDED_ROUTES` if they should work in preview mode, or blocked if preview users should not access them. Recommendation: **block Addepar connection for preview users** (no value in simulating external API connections).

7. Security Considerations

7.1 Credential Security

Concern	Mitigation
API key storage	Encrypted at rest using <code>Crypt::encryptString()</code>
Key in transit	HTTPS only (Addepar enforces this)
Key in logs	Never log API key/secret; mask in error messages
Key in frontend	Never expose to Vue; all API calls server-side
Key rotation	Support re-keying via Settings without data loss

7.2 Data Access Control

- Addepar connection is per-user (not per-household) — each user manages their own connection
- Fynla's existing `auth:sanctum` middleware protects all Addepar endpoints
- Joint owner data: Only sync accounts the authenticated user has permission to view in Addepar
- Admin users cannot access other users' Addepar connections

7.3 Audit Trail

All Addepar sync operations should be logged via Fynla's existing **Auditable** trait:

- Connection created/deleted
- Sync triggered (manual/scheduled)
- Account mappings changed
- Values updated (before/after for audit compliance)

8. Error Handling

Error Scenario	Addepar Response	Fynla Handling
Invalid credentials	401 Unauthorized	Show "Invalid API key" in UI; clear stored credentials
Insufficient permissions	403 Forbidden	Show "Addepar permissions insufficient" with guidance
Entity not found	404 Not Found	Mark mapping as stale; flag for review
Rate limited	429 Too Many Requests	Exponential backoff; queue remaining requests
Timeout (>60s)	Request timeout	Retry with pagination; reduce batch size
Addepar downtime	5xx errors	Retry up to 3 times; show "Addepar unavailable"
Data mismatch	N/A	Log discrepancy; keep Fynla data as fallback
Currency mismatch	Non-GBP currency_factor	Convert using exchange rate or flag for user

Use **FinancialCalculationException** factory methods for domain-specific errors:

```
throw FinancialCalculationException::investmentCalculationError(  
    'Addepar sync failed: ' . $reason,  
    ['entity_id' => $entityId, 'user_id' => $userId]  
);
```

9. Data Conflict Resolution

When Addepar data conflicts with manually-entered Fynla data:

Scenario	Resolution Strategy
Account value differs	Addepar wins (custodial source of truth)
Holdings differ	Addepar wins (positions are authoritative)
Account name differs	Keep Fynla name (user preference)
Account type differs	Keep Fynla classification (user-confirmed mapping)

Scenario	Resolution Strategy
Ownership percentage differs	Flag for user review
Account exists in Fynla but not Addepar	Keep Fynla record (may be non-custodied asset)
Account exists in Addepar but not Fynla	Show in import preview for user to accept/reject
Fee data differs	Use Addepar (actual) over Fynla (estimated)

Principle: Addepar is authoritative for **valuations and positions**. Fynla is authoritative for **planning parameters** (retirement age, risk preferences, goals, tax elections).

10. Migration Path for Existing Users

Phase 1: Connect & Map (Week 1-2)

- 1. User connects Addepar credentials in Settings
- 2. System fetches entity list
- 3. User reviews auto-classification and corrects mappings
- 4. First full sync populates/updates Fynla records

Phase 2: Validate & Reconcile (Week 2-3)

- 5. User reviews imported data against their manual entries
- 6. Resolves any conflicts (guided UI)
- 7. Marks accounts as "Addepar-managed" (auto-sync enabled)

Phase 3: Steady State (Ongoing)

- 8. Daily incremental syncs
- 9. Manual entry only for non-Addepar assets (property, DB pensions, protection)
- 10. Net worth and projections use blended data

11. Limitations & Caveats

Limitation	Impact	Workaround
Addepar is USD-centric	Currency mismatch with Fynla (GBP)	Filter/convert; reject non-GBP accounts or apply FX
Custom attributes vary per firm	Risk profile, age may use different attribute keys	Configurable attribute mapping per connection
No webhooks (push) documented	Cannot receive real-time updates	Polling via scheduled sync
Rate limits undocumented	Risk of throttling during large syncs	Conservative rate limiting; incremental sync
DB pensions not in Addepar	Retirement module partially covered	DB pensions remain manual entry

Limitation	Impact	Workaround
Protection policies not in Addepar	Protection module not covered	Insurance data stays manual
Property not in Addepar	Property module not covered	Property data stays manual
Addepar requires enterprise subscription	Cost barrier for individual users	Position as "Professional" tier feature
60-second request timeout	Large portfolios may timeout	Pagination and batched requests

12. Implementation Estimate

New Files

Backend (PHP):

File	Purpose
app/Services/Addepar/AddeparClient.php	HTTP client with auth, retry, rate limiting
app/Services/Addepar/AddeparSyncService.php	Sync orchestration
app/Services/Addepar/AddeparEntityMapper.php	Entity-to-model mapping
app/Services/Addepar/AddeparPortfolioMapper.php	Holdings/positions mapping
app/Services/Addepar/AddeparTransactionMapper.php	Transaction import
app/Services/Addepar/AddeparAttributeResolver.php	Custom attribute discovery
app/Http/Controllers/Api/AddeparController.php	API endpoints
app/Http/Requests/AddeparConnectRequest.php	Validation
app/Models/AddeparConnection.php	Connection model
app/Models/AddeparAccountMapping.php	Mapping model
app/Console/Commands/AddeparSyncCommand.php	Scheduled sync artisan command
database/migrations/xxxx_create_addepar_connections_table.php	Connection table
database/migrations/xxxx_create_addepar_account_mappings_table.php	Mapping table
database/factories/AddeparConnectionFactory.php	Test factory

File	Purpose
database/factories/AddeparAccountMappingFactory.php	Test factory

Frontend (Vue):

File	Purpose
resources/js/components/Addepar/AddeparConnectionCard.vue	Connect/disconnect UI
resources/js/components/Addepar/AddeparSyncStatus.vue	Sync indicator
resources/js/components/Addepar/AddeparAccountMapping.vue	Mapping interface
resources/js/components/Addepar/AddeparMappingRow.vue	Individual mapping row
resources/js/components/Addepar/AddeparSyncModal.vue	Sync progress
resources/js/components/Addepar/AddeparImportPreview.vue	Import review
resources/js/services/addeparService.js	API service
resources/js/store/modules/addepar.js	Vuex store

Tests:

File	Purpose
tests/Unit/Services/Addepar/AddeparClientTest.php	Client auth & requests
tests/Unit/Services/Addepar/AddeparEntityMapperTest.php	Entity mapping logic
tests/Unit/Services/Addepar/AddeparPortfolioMapperTest.php	Holdings mapping
tests/Unit/Services/Addepar/AddeparSyncServiceTest.php	Sync workflow
tests/Feature/Api/AddeparControllerTest.php	API endpoint tests

13. Configuration

Environment Variables

```
# Addepar Integration
ADDEPAR_ENABLED=false
ADDEPAR_DEFAULT_SYNC_INTERVAL=daily
ADDEPAR_REQUEST_TIMEOUT=55
ADDEPAR_MAX_REQUESTS_PER_MINUTE=60
ADDEPAR_RETRY_ATTEMPTS=3
```

Feature Flag

Gate the entire integration behind `ADDEPAR_ENABLED` in `.env`. This allows:

- Gradual rollout to specific users/tiers

- Easy disable if Addepar API changes
- No impact on users who don't use Addepar

Subscription Tier

Recommended: Restrict Addepar integration to **Pro tier** (£19.99/month) — aligns with the professional/adviser user profile who would have an Addepar subscription.

14. Summary of Fynla Module Coverage

Fynla Module	Addepar Coverage	Data Flow
Investment	Full	Accounts, holdings, valuations, performance, fees
Retirement (DC)	Partial	Pension fund values, holdings, contributions
Retirement (DB)	None	Manual entry only
Savings	Limited	Cash/money market accounts only
Protection	None	Not applicable
Estate	Indirect	Updated investment values flow into IHT calculations
Property	None	Not applicable
Goals	Indirect	Linked account balances auto-update
Net Worth	Indirect	All synced account values feed net worth
Risk	Bidirectional	Import/export risk attributes
Tax	None	Tax config is UK-regulatory, not custodial

15. Next Steps

1. **Addepar sandbox access** — Request a development environment from Addepar to validate API endpoints and response formats
2. **Custom attribute audit** — Document which custom attribute keys are used by target firms for risk, age, and classification
3. **Prototype AddeparClient** — Build and test basic auth + entity fetch against sandbox
4. **Design mapping UI** — Wireframe the account mapping interface for user review
5. **Define sync conflict UX** — Design the reconciliation flow for existing manual data
6. **Security review** — Run the **security-reviewer** agent on the credential storage implementation
7. **Subscription gating** — Implement feature flag and tier restriction